Type Systems That Reinterpret Themselves

ANONYMOUS AUTHOR(S)

We advocate that type systems should be able to reinterpret their own terms in any internal model of themselves: equipped with a single tool of reinterpretation, you can express a variety of features, such as universe polymorphism, external parametricity, ability to do synthetic mathematics internally and more. To this end, we explore limits of self-reinterpretation and present two type systems capable of it, namely HM-R and MLTT-R, prove their basic properties and show several examples of useful internal models.

1 Introduction

Consider the following well-known function which maps over elements of the list, written in Agda:

```
map : ∀ {A B : Set} → (A → B) → List A → List B
map f [] = []
map f (x ∷ xs) = f x ∷ map f xs
```

Enjoying both composability of this function and simplicity of its type, we would like to reuse its definition as much as possible. Up to a certain level, we would surely be able to do so: after all, `map` is polymorphic in source and target types, so we can instantiate it with whatever types we want. However, were we to find a magic lamp with a type-theoretical djinn inside, we would ask him to grant us three wishes:

(1) Reuse `map` at higher universe levels, e.g., given a *type operator*, map over *list of types*:

$$(\mathtt{Set} \to \mathtt{Set}) \to \mathtt{List}\,\mathtt{Set} \to \mathtt{List}\,\mathtt{Set}$$

(2) Automagically obtain a proof of the free theorem for `map` – that is, some term of type

$$\begin{aligned}&\forall A\,A'\,(Q : A \to A' \to \mathtt{Prop}), \forall B\,B'\,(R : B \to B' \to \mathtt{Prop}),\\ &\qquad \forall f\,f', (\forall x\,x', Q\,x\,x' \to R(f\,x)\,(f'\,x')) \to\\ &\qquad \forall (\mathtt{xs} : \mathtt{List}\,A)(\mathtt{xs}' : \mathtt{List}\,A'), Q^*\,\mathtt{xs}\,\mathtt{xs}' \to R^*(\mathtt{map}\,f\,\mathtt{xs})(\mathtt{map}\,f'\,\mathtt{xs}');\end{aligned}$$

(3) Given groupoids A and B and a functor $f : A \to B$, construct a functor `map` f between groupoids `List` A and `List` B.

The first wish is, of course, granted by adjusting the definition and making it level-polymorphic. While the theorem in the second wish already holds in systems with parametricity, obtaining a proof *inside* the system without human input is only possible in type systems where parametricity is internalized [1, 3]. The last wish currently has to be fulfilled manually (or with some metaprogramming involved [13]), albeit with a couple of list inductions: some to define a groupoid instance for `List` and few more to prove functoriality of `map`.

We suggest a different approach. Before tweaking the definition of `map`, let's first remember all of the metatheoretic power our type system already has. Abovementioned parametricity of polymorphic definitions is the first result that comes to mind; note that it is obtained by specifying a *relational interpretation* of the typing judgement [14]. And indeed, ones of the most interesting metatheoretic results can be expressed by constructing a certain *model* of a type system: synthetic results in homotopy theory are based on $((\infty, 1)$-)groupoid model of (homotopy) type theory [2, 7]; McBride's usage of displacement for universe polymorphism [10, 11] can be viewed as constructing a model of a type system in higher universes of itself and then interpreting displaced constructions in it. Then, all of our wishes listed above could be granted by constructing a suitable model of a type system *inside* itself (hence we call these *internal models*) and then instantiating `map` in it. Throughout the paper, we'll call type systems with self-reinterpretation capabilities *reflective*.

1.1 Contributions

- Our main goal is to construct an implementable reflective dependently typed system. Main difficulty in doing so arises from a certain no-go result discussed in section 2.1 which forces us to limit the power of reinterpretation in order to keep type checking decidable.
- For ease of presentation, we first offer a reflective type system based on Hindley-Milner type system called HM-R (section 3). We prove that HM-R is sound, conservative with respect to simply-typed lambda calculus and that global type inference is decidable.
- As a warmup exercise, we provide an internal model of HM-R (section 3.2) which by Curry-Howard correspondence translates into hypothesis elimination in intuitionistic logic.
- Building upon the insights gathered from developing HM-R, we're finally equipped to dive into reflective version of Martin-Löf type theory dubbed MLTT-R (section 4). We prove soundness and conservativity of MLTT-R as well.
- As intended, by virtue of reinterpretation, MLTT-R incorporates both a restricted form of universe polymorphism and external parametricity. And, of course, you can use MLTT-R as a system for synthetic mathematics with internal reuse of synthetic results (section 4.2).

2 Main ideas

We aim to design a type system that allows its users to reuse native syntax of a system for special kinds of user-defined "domain-specific languages". This approach has several advantages:

- From the pragmatic standpoint, it would allow DSL designers to use a syntax which is already familiar to their users;
- From type-theoretical standpoint, type theorists could internalize metatheoretical properties of such type system directly inside itself;
- From philosophical standpoint, it would be particularly satisfying to have a type system capable of "eating itself".

To define a new DSL in such system, it is enough to specify how to interpret the native syntax inside your domain of interest. It is natural to expect that provided interpretations respect definitional equalities of a system; finally, we want interpretations to be definable inside a system, so we call them *internal models* of a system, or just *models* for short.

Now, before actually specifying a system, let's find out what we cannot hope to achieve.

2.1 No-go theorem of internal interpretation

We're going to work inside a classical type-theoretical framework which introduces a sort of contexts Γ, terms t, t' and types T linked by the typing relation $\Gamma \vdash t : T$; computation is specified by one-step reduction relation $t \longrightarrow t'$ on terms. In this section we'll show that, if we want a practical system with internal interpretation as discussed in the start of the section 2, we have to either compute reinterpretation only on closed terms, or severely restrict a class of definable models.

Definition 2.1 (Naïve computational reinterpretation). We call a type operator $F[_]$ a naïve computational reinterpretation of types if it respects computation on types (that is, if there is a reduction relation on types ($\longrightarrow$) and $T \longrightarrow T'$, then $F[T] \longrightarrow^* F[T']$), and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a naïve computational reinterpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Theorem 2.2 (No-go theorem of naive reinterpretation). *In a confluent type system with a type of functions and naïve computational reinterpretation, if reduction commutes with substitution,*

for arbitrary well-typed s and t there exists u such that:

$$\mathsf{app}[\![\lambda x.t]\!][\![s]\!] \to^* u \ {}^*\!\leftarrow [\![t[x \coloneqq s]]\!]$$

where app *is how* $[\![_]\!]$ *acts on applications.*

Principally, it means that **propositional equality is not strong enough to enforce equational properties of models**. The proof is by direct example:

Proof. We are given well-typed $\Gamma \vdash s : S$ and $\Gamma, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : S \to T, y : S \vdash [\![x\, y]\!] : F[T]$$

Term under interpretation is in normal form, hence the whole term computes to $\mathsf{app}[\![x]\!][\![y]\!]$. Substituting $\lambda x.t$ for x and s for y, we get $\mathsf{app}[\![\lambda x.t]\!][\![s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![(\lambda x.t)s]\!] : F[T]$. Since naïve reinterpretation respects reduction, this reduces to $[\![t[x \coloneqq s]]\!]$.

Since reduction commutes with substitution and system is confluent, there should be such u that both terms reduce to. □

Fundamentally, the proof above exploits openness of the original term; if we restrict reinterpretation to compute on closed terms only, we can arrive at a sound system without requiring definitional equalities in models, as is exemplified by our non-dependently-typed system STLC-R described in section 3. However, such a tradeoff is unacceptable in a system with dependent types which has to compute on open terms in order to have a flexible typing discipline. This leaves us with three options:

(1) Introduce a "modality of global elements" □ which reifies a metatheoretical notion of a closed term in a dependent type system, then allow to reinterpret boxed terms only.
(2) Forbid computations under $[\![_]\!]$ unless term is closed. To allow reasoning on open terms, equations describing behavior of interpretation are exposed as axioms in equality type.
(3) Restrict models to those where equalities hold definitionally rather than propositionally.

While the first approach is most desirable, it unfortunately still doesn't work, as is shown in section 5.1.1. The second one looks consistent with propositional definition of models and is left for future work; in 4, we explore the third approach as it seems easier to work with in practice thanks to more flexible computation rules.

3 Almost simply typed reinterpretable language: STLC-R

We start with a simpler language to iron out the details and actually figure out how to properly reinterpret terms internally before moving on to dependently typed system. Our starting point is simply typed lambda calculus with products, which we call «STLC» throughout this paper. As discussed before, we will add self-reinterpretation capabilities to it to enable reinterpretation in any «model» of STLC specifiable inside our language. Here's the thing we *wish* to do with it, where $t \mathbin{@} \mathcal{M}$ is an operator of reinterpretation of term t in internal model $\mathcal{M}$:

$$(\lambda f x y.(f x, y)) \mathbin{@} \langle (\circ), \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \mathtt{lam}, \mathtt{eval}, \ldots \rangle =$$
$$= \mathtt{lam}\,(\mathtt{lam}\,(\mathtt{lam}\,(\mathtt{pair}\,(\mathtt{eval} \circ \mathtt{pair}\,(\mathtt{proj}_2 \circ \mathtt{proj}_1 \circ \mathtt{proj}_1)\,(\mathtt{proj}_2 \circ \mathtt{proj}_1))\,\mathtt{proj}_2)))$$

Where `lam`, `pair` and `eval` are correspondingly constructor of an exponential object, constructor of a product and evaluation morphism, in the same way they are specified in the definition of cartesian closed categories.

There are a few things to ponder:

- How to reinterpret free variables?

MonoType	S, T, C, P, E, TT	$::=$	α	type variable
		$\mid$	$\top$	unit type
		$\mid$	$S \times T$	product type
		$\mid$	$S \to T$	arrow type
PolyType	σ	$::=$	T	monomorphic type
		$\mid$	$\forall \alpha.\sigma$	forall type
Term	s, t	$::=$	x	variable
		$\mid$	$()$	singleton
		$\mid$	(s, t)	pair
		$\mid$	$\pi_1\ t$	first projection
		$\mid$	$\pi_2\ t$	second projection
		$\mid$	$\lambda x.t$	abstraction
		$\mid$	$t\ s$	application
		$\mid$	$\texttt{let}\ x = s\ \texttt{in}\ t$	polymorphic binding
		$\mid$	$x^{\mathcal{M}}$	reinterpretation
Model	$\mathcal{M}$	$::=$	$\langle T, t, \ldots \rangle$	model def. (see below)

Fig. 1. STLC-R grammar

- How to type universes?
- We're in a simply typed setting, so there's no way to enforce that universe satisfies coherence conditions on their operations. How to structure computation in such a way that it doesn't break Church-Rosser property?

To answer the first question, we actually disallow (most of) free variables in a terms that are to be reinterpreted since it is not clear how to lift arbitrary *value* into current universe, so we have to enforce some kind of a typing discipline for introducing terms amenable to reinterpretation. In a sense, reinterpretation is a strong form of polymorphism, so let's structure our grammar and typing rules similarly to how it's done in Hindley-Milner type systems today and see how far we can go; grammar rules are collected in a figure 1.

In figure 2, we mostly repeat usual declarative typing rules of a Hindley-Milner type system [5] (TODO: check source) with an addition of a rule for reinterpretation (discussed below) and an important change in `let` case: as discussed, we don't take local context in account when checking polymorphic term s. Instead, it can only reference other polymorphic terms. This is safe since other polymorphic terms are constrained in the same way, and can be safely used inside s.

To check models, we abuse our type system a bit in order to avoid adding type operators and complicate the type system: types of arrows C, products P and exponentials E are specified as open monomorphic types which can use two type variables, 0 and 1. This fact is reflected in the syntax of a model using notation $\Lambda 01.C$, $\Lambda 01.P$ and $\Lambda 01.E$ respectively. 0 and 1 are then filled in by substitution when we need to type model operations (notation $T[S_1, S_2]$ is a shorthand for $T[0 \coloneqq S_1, 1 \coloneqq S_2]$).

We could generalize model definition to include something else as objects instead of types, but there is no way to build meaningful C,P and E with anything other than types in base system.

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \qquad \Gamma \vdash () : \top \qquad \frac{\Gamma \vdash s : S \qquad \Gamma \vdash t : T}{\Gamma \vdash (s, t) : S \times T} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_1\, t : S} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_2\, t : T}$$

$$\frac{\Gamma, x : S \vdash t : T}{\Gamma \vdash \lambda x.t : S \to T} \qquad \frac{\Gamma \vdash t : S \to T \qquad \Gamma \vdash s : S}{\Gamma \vdash t\, s : T} \qquad \frac{\Gamma \vdash t : \sigma}{\Gamma \vdash t : \forall \alpha.\sigma} \qquad \frac{\Gamma \vdash t : \forall \alpha.\sigma}{\Gamma \vdash t : \sigma[\alpha \coloneqq T]}$$

$$\frac{\mathrm{fv}(\sigma) = \varnothing \qquad \forall \Gamma \vdash s : \sigma \qquad \forall \Gamma, x : \sigma, \Delta \vdash t : T}{\forall \Gamma, \Delta \vdash \mathtt{let}\, x = s\, \mathtt{in}\, t : T} \qquad \frac{x : \forall \vec{\alpha}.T \in \forall \Gamma \qquad \forall \Gamma, \Delta \vdash \mathcal{M}\,\mathrm{model}}{\forall \Gamma, \Delta \vdash x^{\mathcal{M}} : [\![T]\!]^{\mathcal{M}}_{[\vec{\alpha} \coloneqq \vec{S}]}}$$

$$\frac{\begin{array}{c}\mathrm{fv}(C) \cup \mathrm{fv}(P) \cup \mathrm{fv}(E) \subseteq \{0, 1\} \\ \Gamma \vdash \mathtt{id} : \forall \alpha.C[\alpha, \alpha] \qquad \Gamma \vdash \mathtt{comp} : \forall \alpha\beta\gamma.C[\beta, \gamma] \to C[\alpha, \beta] \to C[\alpha, \gamma] \\ \mathrm{fv}(TT) = \varnothing \qquad \Gamma \vdash \mathtt{cnst} : \forall \alpha.C[\alpha, TT] \qquad \Gamma \vdash \mathtt{pair} : \forall \alpha\beta\gamma.C[\gamma, \alpha] \to C[\gamma, \beta] \to C[\gamma, P[\alpha, \beta]] \\ \Gamma \vdash \mathtt{proj}_1 : \forall \alpha\beta.C[P[\alpha, \beta], \alpha] \qquad \Gamma \vdash \mathtt{proj}_2 : \forall \alpha\beta.C[P[\alpha, \beta], \beta] \\ \Gamma \vdash \mathtt{lam} : \forall \alpha\beta\gamma.C[P[\gamma, \beta], \alpha] \to C[\gamma, E[\alpha, \beta]] \qquad \Gamma \vdash \mathtt{eval} : \forall \alpha\beta.C[P[E[\alpha, \beta], \beta], \alpha]\end{array}}{\Gamma \vdash \langle \Lambda 01.C, \mathtt{id}, \mathtt{comp}, TT, \mathtt{cnst}, \Lambda 01.P, \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \Lambda 01.E, \mathtt{lam}, \mathtt{eval}\rangle\,\mathrm{model}}$$

Fig. 2. STLC-R declarative typing rules

$$u, v ::= n \mid () \mid (u, v) \mid \lambda x.v$$
$$n ::= x \mid \pi_i\, n \mid n\, v$$

Fig. 3. Values and neutral terms of STLC-R

Definition 3.1. Let $\mathcal{M} = \langle \Lambda 01.C, \mathtt{id}, \mathtt{comp}, TT, \mathtt{cnst}, \Lambda 01.P, \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \Lambda 01.E, \mathtt{lam}, \mathtt{eval}\rangle$. Then, interpretation of type in a model is the following function:

$$[\![T]\!]^{\mathcal{M}}_{\sigma} = C[TT, T^{\mathcal{M}}_{\sigma}] \tag{1}$$
$$\alpha^{\mathcal{M}}_{\sigma} = \sigma\alpha \tag{2}$$
$$\top^{\mathcal{M}}_{\sigma} = TT \tag{3}$$
$$(S \times T)^{\mathcal{M}}_{\sigma} = P[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}] \tag{4}$$
$$(S \to T)^{\mathcal{M}}_{\sigma} = E[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}]. \tag{5}$$

Having defined typing rules, let's pin down computation rules in order to prove some useful properties. General idea is the following: before inlining a `let` binding, we first have to make sure that the polymorphic term is closed and is in normal form; in all other cases, standard reductions of STLC apply.

$$\frac{s \to s'}{(s,t) \to (s',t)} \qquad \frac{t \to t'}{(s,t) \to (s,t')} \qquad \frac{t \to t'}{\pi_i t \to \pi_i t'} \qquad \pi_i(t_1,t_2) \to t_i$$

$$\frac{t \to t'}{\lambda x.t \to \lambda x.t'} \qquad \frac{s \to s'}{s\,t \to s'\,t} \qquad \frac{t \to t'}{s\,t \to s\,t'} \qquad (\lambda x.t)s \to t[x \coloneqq s]$$

$$\frac{s \to s'}{\texttt{let}\,x = s\,\texttt{in}\,t \to \texttt{let}\,x = s'\,\texttt{in}\,t} \qquad \frac{t \to t'}{\texttt{let}\,x = s\,\texttt{in}\,t \to \texttt{let}\,x = s\,\texttt{in}\,t'}$$

$$\frac{\mathrm{fv}(v) = \varnothing}{\texttt{let}\,x = v\,\texttt{in}\,t \to t[\![x \coloneqq v]\!]}$$

Fig. 4. STLC-R computation rules

Definition 3.2. By $t[\![x \coloneqq v]\!]$ we mean an operation which works as a regular substitution of v into t except for the case of reinterpretation:

$$x^{\mathcal{M}}[\![x \coloneqq v]\!] = [\![v]\!]^{\mathcal{M}}/\varepsilon \tag{6}$$

$$[\![()]\!]^{\mathcal{M}}/\gamma = \texttt{cnst} \tag{7}$$

$$[\![(s,t)]\!]^{\mathcal{M}}/\gamma = \texttt{pair}\,[\![s]\!]^{\mathcal{M}}/\gamma\,[\![t]\!]^{\mathcal{M}}/\gamma \tag{8}$$

$$[\![\pi_i\,t]\!]^{\mathcal{M}}/\gamma = \texttt{comp}\,\texttt{proj}_i\,[\![t]\!]^{\mathcal{M}}/\gamma \tag{9}$$

$$[\![x]\!]^{\mathcal{M}}/\gamma = \gamma\,x \tag{10}$$

$$[\![\lambda x.t]\!]^{\mathcal{M}}/\gamma = \texttt{lam}\,[\![t]\!]^{\mathcal{M}}/[\vec{y} \coloneqq \texttt{comp}\,\texttt{proj}_1(\gamma\,\vec{y}),\ x \coloneqq \texttt{proj}_2] \tag{11}$$

$$[\![t\,s]\!]^{\mathcal{M}}/\gamma = \texttt{comp}\,\texttt{eval}\,\left(\texttt{pair}\,[\![t]\!]^{\mathcal{M}}/\gamma\,[\![s]\!]^{\mathcal{M}}/\gamma\right) \tag{12}$$

where model operations $\texttt{cnst}, \texttt{pair}$ and others are taken from $\mathcal{M}$ and ε is an empty substitution.

3.1 Metatheoretical properties of STLC-R

Here we only study some basic properties of a system to make sure that everything works as expected: computations don't diverge; types are preserved by all computations; all of the types can be inferred if you wish to do so; everything terminates and, modulo reduction of `let`-bindings, this is the same old STLC we started with. Proofs are mostly following standard techniques, since rules for reinterpretation are structured to directly plug into proofs of properties of STLC.

Theorem 3.3 (Church-Rosser theorem for STLC-R). *Reflexive transitive closure of* $(\to)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [15]. Parallel reduction is defined as:

$$t \twoheadrightarrow t \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(s,t) \twoheadrightarrow (s',t')} \qquad \frac{t \twoheadrightarrow t'}{\pi_i\, t \twoheadrightarrow \pi_i\, t'} \qquad \frac{t_1 \twoheadrightarrow t_1' \qquad t_2 \twoheadrightarrow t_2'}{\pi_i(t_1,t_2) \twoheadrightarrow t_i'}$$

$$\frac{t \twoheadrightarrow t'}{\lambda x.t \twoheadrightarrow \lambda x.t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{s\, t \twoheadrightarrow s'\, t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(\lambda x.s)\, t \twoheadrightarrow s'[x \coloneqq t']}$$

$$\frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{\mathtt{let}\, x = s\, \mathtt{in}\, t \twoheadrightarrow \mathtt{let}\, x = s'\, \mathtt{in}\, t'} \qquad \frac{t \twoheadrightarrow t'}{\mathtt{let}\, x = v\, \mathtt{in}\, t \twoheadrightarrow t'[\![x \coloneqq v]\!]}$$

□

Theorem 3.4 (Type preservation for STLC-R). *The following is true:*

(1) *If* $\mathcal{M}$ model, $\gamma\, x : C[\Gamma^{\mathcal{M}}, \Gamma(x)^{\mathcal{M}}]$ *for all* $x \in \Gamma$ *and* $\Gamma \vdash t : T$, *then* $[\![t]\!]^{\mathcal{M}}/\gamma : C[\Gamma^{\mathcal{M}}, T^{\mathcal{M}}]$.
(2) *If* $\Gamma \vdash s : S$ *and* $\Gamma, x : S, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[x \coloneqq s] : T$.
(3) *If* $\Gamma \vdash v : \sigma$ *and* $\Gamma, x : \sigma, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[\![x \coloneqq v]\!] : T$.
(4) *If* $\Gamma \vdash t : T$ *and* $t \longrightarrow t'$, *then* $\Gamma \vdash t' : T$.

Proof. By structural induction on terms and $(\longrightarrow)$. □

Theorem 3.5 (Strong normalization for STLC-R). *Closed terms typable in STLC-R are strongly normalizing with respect to* $(\longrightarrow)$.

Proof. By Tait's method of logical relations. Let SN be a set of strongly normalizing terms. Let there be the following relation $\mathcal{R} \subseteq \mathsf{PolyType} \times \mathsf{Term}$:

- $t \in \mathcal{R}_\alpha, \mathcal{R}_\top$ iff $t \in SN$;
- $t \in \mathcal{R}_{T_1 \times T_2}$ iff $\pi_i\, t \in \mathcal{R}_{T_i}$ for $i = 1, 2$;
- $t \in \mathcal{R}_{S \to T}$ iff for all $s \in \mathcal{R}_S$, $t\, s \in \mathcal{R}_T$;
- $t \in \mathcal{R}_{\forall \vec{\alpha}.T}$ iff $t \in SN_{\mathtt{let}}$ and for all $\mathcal{U} \in \mathcal{R}_{univ}$ and $\vec{S}$, $nf_{\mathtt{let}}(t)\ @\ \mathcal{U} \in \mathcal{R}_{\mathcal{U}[\![T]\!][\vec{\alpha} \coloneqq \vec{S}]}$.

$\mathcal{U} \in \mathcal{R}_{univ}$ is defined similarly by requiring that all terms inside $\mathcal{U}$ belong to respective $\mathcal{R}$-s.

Then we can prove our main lemma by mutual induction on typing rules:

- If $\Gamma \vdash t : T$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}_{\Gamma(x)}$ for all $x \in \Gamma$ then $\gamma\, t \in \mathcal{R}_T$.
- If $\forall \Gamma \vdash t : T$, $\vec{\alpha} = fv(T)$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}_{\Gamma(x)}$ for all $x \in \Gamma$ then $\gamma\, t \in \mathcal{R}_{\forall \vec{\alpha}.T}$.

Noticing $\mathcal{R}_T \subseteq SN$ completes the proof. □

Theorem 3.6 (Conservativity over STLC). *Normal forms of closed terms typable in STLC-R are typable in STLC.*

Proof. Thanks to type preservation, normal form $nf(t)$ is typable in STLC-R. Normal form cannot contain let-bindings by definition of $(\longrightarrow)$, therefore each subterm is $\forall$-closed. Therefore $nf(t)$ has no reinterpretation subterms. □

Theorem 3.7. *Global type inference for STLC-R is decidable.*

Proof. It is enough to extend any unification-based type inference algorithm for Hindley-Milner type system to support universes and reinterpretation.

- Since universes are provided explicitly, it is enough to *check* them.
- Since polymorphic functions do not depend on local context, their type is fully resolved when it's added to context.

- Once universe is checked in $x \mathbin{@} \mathcal{U}$, we can directly compute $\mathcal{U}[\![T]\!]$ (since $\mathcal{U}$ is known and T is fully resolved) and instantiate it with fresh metavariables in $\vec{S}$.

□

3.2 Useful reinterpretations inside STLC-R

The idea to interpret STLC terms in cartesian closed categories is really not new and is already thoroughly explored by Conal Elliott in "Compiling to Categories" [6]; our addition here is that we internalized reinterpretation inside the calculus without working through compiler plugin. So all of the main applications from the cited paper are still applicable here whenever they are expressible as universes in STLC-R. As STLC-R is not the main focus of the paper, we will not be reiterating the examples here, but we would like to add just one more example.

Following Moggi [12], we can use monads to model side effects in a language. What is new here is that instead of parameterizing a function by arbitrary monad, we can simply *reinterpret* pure terms as if they were run in monadic context. Therefore, our universe is a Kleisli category; however, it is only cartesian, but in general not cartesian *closed*. The problem is the following operation (here F is a monad in question, E is whatever exponential object we were to use):

$$\texttt{lam} : (\alpha \times \beta \to F[\gamma]) \to (\alpha \to F[E[\gamma, \beta]])$$

In other words, F has to *distribute* over an arrow. Well, at least arrow type distributes over itself:

$$\begin{aligned}
\mathcal{U}_E = \langle \\
&\Lambda 01.\, 0 \to E \to 1, \\
&\lambda x e.x, \\
&\lambda g f x e.g(fxe)e, \\
&\top, \\
&\lambda e.(), \\
&\Lambda 01.\, 0 \times 1, \\
&\lambda f g x e.(fxe, gxe), \\
&\lambda p e.\pi_1\, p, \\
&\lambda p e.\pi_2\, p, \\
&\Lambda 01.\, 0 \to 1, \\
&\lambda f x e y.f(x, y)e, \\
&\lambda p e.(\pi_1\, p)\ (\pi_2\, p) \\
\rangle
\end{aligned}$$

Interestingly enough, if, using Curry-Howard correspondence, we take closed STLC term t to represent proof of an intuitionistic proposition, $t \mathbin{@} \mathcal{U}_E$ corresponds to hypothesis elimination in intuitionistic logic.

We *could* write down a universe for arbitrary monad if we had capability to interpret terms which do not use abstractions in arbitrary cartesian category, which is actually achievable with [6]. We leave study of such "partial" interpretation for future work.

4 Dependently typed reinterpretable language: MLTT-R

Now we are ready to take on the main challenge: design a minimal dependently typed reinterpretable language, and provide internal models of itself to yield universe polymorphism, external

parametricity and do synthetic mathematics inside it. As a starting point, we take Martin-Löf type theory with $\Pi-$,$\Sigma-$, Id$-$ types, natural number type and infinite non-cumulative hierarchy of universes. The problems are similar to the ones we had with simply-typed calculus:

- How to reinterpret free variables?
- How to type universes?
- Now, there is a way to enforce that internal model satisfies coherence conditions on their operations. How to structure computation in such a way that it doesn't break Church-Rosser property?
- How to respect conversion rule?

As before, we sidestep the question with free variables by considering a term for reinterpretation iff it is closed in local context. To properly capture this property, we introduce a modality of closed terms $\Box$ which is already extensively studied in previous works (TODO: provide refs). To type universes, we have to consider:

- Typing rules for MLTT are necessarily mutually recursive with rules of context well-formedness. Which framework to use to capture this in a model?
- How to provide a "universe" for each level in the interpreted term?

To deal with the first problem, we use Categories-with-Families [4] as one of the most convenient models of dependent type theories in use today. Second one, however, is reminiscent of classical circularity paradoxes in logic and computer science: if we were to provide a model to each universe at each finite level, in which universe does the model lie? The first solution which comes to mind is to add another sequence of levels $\omega, \omega + 1, \ldots$ and allow to use only finite-level universes inside reinterpretable term; such a system, however, would not "eat itself", a property that we wanted to achieve. Consider instead the following observation: in a system without level binders, each term uses only a finite number of levels. Hence to reinterpret this term, we need to only provide a finite number of universe models, which can together fit into some universe of a finite level.

The problem with Church-Rosser is solved in absolutely the same way as with simply-typed case: we just do not perform any β-reductions inside a let-binding, only inline `let` declarations before performing a substitution.

The problem with conversion rule is the following: in definition of a universe, we can now ask for propositional proofs of coherence. Thus types which were *definitionally* equal before reinterpretation are now only *propositionally* equal. To solve this issue, we start by first pinpointing the locations where conversion rule is actually used; in order to do this, we *need* to use bidirectional type checking. After translation, point of conversion is wrapped in a `rewrite` call.

But how do we synthesize a propositional proof of equivalence? Here we translate a chain of reductions performed during checking of definitional equality into a chain of propositional equivalences taken from coherence conditions proven in the definition of a model.

To actually compute interpretation in MLTT-R, we need two mutually recursive functions: $t \mathbin{@}^{\gamma}_{\Leftarrow T} \mathcal{U}$ and $t \mathbin{@}^{\gamma}_{\Rightarrow} \mathcal{U}$ for checkable and synthesizing terms, respectively. TODO: FINISH.

4.1 Metatheoretical properties of MLTT-R

Once again, we only try to make sure of the most essential properties: that computations do not diverge, that types during computations are preserved, all computations terminate and that resulting type system is a conservative extension of MLTT.

Theorem 4.1 (Church-Rosser theorem for STLC-R). *Reflexive transitive closure of* $(\rightarrow)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [15]. $\Box$

Context	Γ	::=	ε	empty global context
		|	$\Gamma;\Theta$	localization
Small context	Θ	::=	$\cdot$	empty small context
		|	$\Theta, x : T$	variable extension
Type	T, S	::=	$\mathcal{U}_\ell$	universe
		|	$\mathrm{El}_\ell(t)$	universal type family
		|	Nat_ℓ	natural number type
		|	$(x : S) \times T$	Σ-type
		|	$(x : S) \to T$	Π-type
		|	$t_1 =_T t_2$	identity type
		|	$\Box\, T$	type of closed terms
		|	$\mathcal{M}_{\ell_0,\ldots,\ell_\kappa}$	type of models
		|	T^t_Γ	typal reinterpretation
Level	ℓ, κ, j	$\in$	$\mathbb{N}$	finite levels
Term	t, s	::=	$x \mid y \mid z \mid \mathtt{m} \mid \ldots$	variables
		|	U_ℓ	code of a universe
		|	$\mathtt{Nat}_\ell$	code of a natural number type
		|	0	zero
		|	$\mathsf{S}\, t$	successor
		|	$\mathrm{natind}_{x.T}\, t\, \{s_0, y \mapsto s_S\}$	induction
		|	$\mathtt{Sigma}(x : s)t$	code of a Σ-type
		|	(s, t)	pairing
		|	$\pi_1\, t \mid \pi_2\, t$	projections
		|	$\mathtt{Pi}(x : s)t$	code of a Π-type
		|	$\lambda x.\, t$	abstraction
		|	$t\, s$	application
		|	$\mathtt{Id}\, t\, t_1\, t_2$	code of an identity type
		|	refl	reflexivity
		|	$\mathrm{J}_{xy.S}\, t\, s$	J rule (elimination of equality type)
		|	$\mathtt{Box}\, t$	code of a box type
		|	$[t]_{🔒}$	locking
		|	$[t]_{🔓}$	unlocking
		|	$[\![t]\!]^{\ell_0,\ldots,\ell_\kappa}$	internal interpretation
		|	t^s_Γ	intermediate interpretation
		|	$t : T$	ascription

Fig. 5. MLTT-R grammar

Theorem 4.2 (Type preservation for MLTT-R).

Proof. By structural induction on terms and $(\to)$. □

Theorem 4.3 (Strong normalization for MLTT-R). *Closed terms typable in MLTT-R are strongly normalizing with respect to* $(\to)$.

Proof. Tait's method of logical relations for MLTT with modifications similar to ones performed in a proof of strong normalization for STLC-R should suffice. However, we're not completely sure. □

$$\textsc{WF-Loc}\ \frac{\Gamma \vdash \Theta}{\Gamma; \Theta\ \mathrm{wf}} \qquad \textsc{WF-Eps}\ \varepsilon \vdash \cdot \qquad \textsc{WF-Nil}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \cdot} \qquad \textsc{WF-Var}\ \frac{\Gamma \vdash \Theta \qquad \Gamma; \Theta \vdash \mathrm{level}(T) = \ell}{\Gamma \vdash \Theta, x : T}$$

$$\textsc{Level-Uni}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}(\mathcal{U}_\ell) = \ell + 1} \qquad \textsc{Level-El}\ \frac{\Gamma \vdash t \Leftarrow \mathcal{U}_\ell}{\Gamma \vdash \mathrm{level}(\mathrm{El}_\ell(t)) = \ell} \qquad \textsc{Level-Nat}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}(\mathrm{Nat}_\ell) = \ell} \qquad \textsc{Level-Box}\ \frac{\Gamma; \cdot \vdash \mathrm{level}(T) = \ell}{\Gamma \vdash \mathrm{level}(\Box\, T) = \ell}$$

$$\textsc{Level-Sigma, Level-Pi}\ \frac{\Gamma; \Theta \vdash \mathrm{level}(S) = \kappa \qquad \Gamma; \Theta, x : S \vdash \mathrm{level}(T) = \ell}{\Gamma; \Theta \vdash \mathrm{level}((x : S) \times T) = \mathrm{level}((x : S) \to T) = \max(\kappa, \ell)}$$

$$\textsc{Level-Id}\ \frac{\Gamma \vdash \mathrm{level}(T) = \ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow T}{\Gamma \vdash \mathrm{level}(t_1 =_T t_2) = \ell} \qquad \textsc{Level-Model}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{level}\left(\mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) = \max(\ell_0, \ldots, \ell_\kappa) + 1}$$

$$\textsc{Level-TyInterp}\ \frac{\Gamma \vdash t \Rightarrow \mathcal{M}_{\ell_0,\ldots,\ell_\kappa} \qquad \Gamma \vdash \Theta \qquad \Gamma; \Theta \vdash \mathrm{level}(T) = j \qquad j \leqslant \kappa}{\Gamma \vdash \mathrm{level}\left(T^t_\Delta\right) = \ell_j}$$

Fig. 6. Context well-formedness & level placement rules

Theorem 4.4 (Conservativity over MLTT). *For MLTT context* Γ*, MLTT type* T *and MLTT-R term* t*, if* $\Gamma \vdash t \Leftarrow T$*, then* $\Gamma \vdash_{\mathrm{MLTT}} \mathrm{nf}(t) : T$.

Proof. Thanks to type preservation, normal form $\mathrm{nf}(t)$ is typable in MLTT-R. TODO TODO TODO. Therefore $\mathrm{nf}(t)$ is an MLTT term and is typable in MLTT. □

4.2 Useful reinterpretations inside MLTT-R

Here we list some of the language features which reinterpretation enables without any additional language support.

4.2.1 Crude, but effective stratification. Due to McBride [**?**], a technique of crude, but effective stratification works as follows: instead of providing complete support for level polymorphism, let's consider a *displacement operator* which, given a term t using universes $U_i, U_j, U_k, \ldots$, produces a term which is equal to t except for universes which are replaced with $U_{i+1}, U_{j+1}, U_{k+1}, \ldots$. The displaced term is also well-typed, however it is defined «one level higher». Notice that displacement operator is *definable* in MLTT-R as $_ @ \mathcal{U}_{\mathrm{McBride}}$ where

$$\mathcal{U}_{\mathrm{McBride}} = \langle \Lambda l. U_{suc\, l}, \ldots \rangle$$

4.2.2 Displayed type theory.

4.2.3 Internal external parametricity. Demonstrated by Reynolds [14], parametricity is a property of a type system that, in simple words, «polymorphic function cannot meaningfully depend on generic inputs». Thus any polymorphic function of type σ satisfies a certain «free theorem» $[\![\sigma]\!]$ defined via relational interpretation of a type.

$$\textsc{Infer-Uni}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathsf{U}_\ell \Rightarrow \mathcal{U}_{\ell+1}} \qquad \textsc{Infer-Nat}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathsf{Nat}_\ell \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Zero}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash 0 \Leftarrow \mathrm{Nat}_\ell} \qquad \textsc{Check-Succ}\ \frac{\Gamma \vdash t \Leftarrow \mathrm{Nat}_\ell}{\Gamma \vdash \mathsf{S}\,t \Leftarrow \mathrm{Nat}_\ell}$$

$$\textsc{Infer-NatInd}\ \frac{\Gamma \vdash t \Rightarrow \mathrm{Nat}_\ell \qquad \Gamma, x : \mathrm{Nat}_\ell \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash s_0 \Leftarrow T[x \coloneqq 0] \qquad \Gamma, y : \mathrm{Nat}_\ell, z : T[x \coloneqq y] \vdash s_S \Leftarrow T[x \coloneqq \mathsf{S}\,y]}{\Gamma \vdash \mathrm{natind}_{x.T}\ t\ \{s_0, yz.s_S\} \Rightarrow T[x \coloneqq t]}$$

$$\textsc{Infer-Sigma}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Sigma}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}} \qquad \textsc{Check-Pair}\ \frac{\Gamma \vdash s \Leftarrow S \qquad \Gamma \vdash t \Leftarrow T[x \coloneqq s]}{\Gamma \vdash (s, t) \Leftarrow (x : S) \times T}$$

$$\textsc{Infer-Proj}_1\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_1\,t \Rightarrow S} \qquad \textsc{Infer-Proj}_2\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_2\,t \Rightarrow T[x \coloneqq \pi_1\,t]} \qquad \textsc{Infer-Pi}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Pi}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}}$$

$$\textsc{Check-Abs}\ \frac{\Gamma, x : S \vdash t \Leftarrow T}{\Gamma \vdash \lambda x.\,t \Leftarrow (x : S) \to T} \qquad \textsc{Infer-App}\ \frac{\Gamma \vdash t \Rightarrow (x : S) \to T \qquad \Gamma \vdash s \Leftarrow S}{\Gamma \vdash t\,s \Rightarrow T[x \coloneqq s]}$$

$$\textsc{Infer-Id}\ \frac{\Gamma \vdash t \Rightarrow \mathcal{U}_\ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow \mathrm{El}_\ell(t)}{\Gamma \vdash \mathsf{Id}\,t\,t_1\,t_2 \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Refl}\ \frac{\Gamma \vdash t_1 \equiv t_2 : T}{\Gamma \vdash \mathrm{refl} \Leftarrow t_1 =_T t_2}$$

$$\textsc{Infer-J}\ \frac{\Gamma \vdash t \Rightarrow t_1 =_T t_2 \qquad \Gamma, x : T, y : t_1 =_T x \vdash \mathrm{level}(S) = _ \qquad \Gamma \vdash s \Leftarrow S[x \coloneqq t_1, y \coloneqq \mathrm{refl}]}{\Gamma \vdash \mathrm{J}_{xy.S}\,t\,s \Rightarrow S[x \coloneqq t_2, y \coloneqq t]}$$

$$\textsc{Infer-Var}\ \frac{🔒 \notin \Delta}{\Gamma, x : T, \Delta \vdash x \Rightarrow T} \qquad \textsc{Infer-Box}\ \frac{\Gamma.🔒 \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \mathsf{Box}\,t \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Lock}\ \frac{\Gamma.🔒 \vdash t \Leftarrow T}{\Gamma \vdash [t]_🔒 \Leftarrow \Box T} \qquad \textsc{Infer-Unlock}\ \frac{\Gamma^{🔓} \vdash t \Rightarrow \Box T}{\Gamma \vdash [t]_{🔓} \Rightarrow T}$$

$$\textsc{Infer-Interp}\ \frac{\Gamma \vdash t \Rightarrow \Box T \qquad \Gamma.🔒 \vdash \mathrm{level}(T) = \kappa}{\Gamma \vdash [\![t]\!]^{\ell_0,\ldots,\ell_\kappa} \Rightarrow \left(\mathsf{m} : \mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) \to T.^{\mathsf{m}}} \qquad \textsc{Infer-Interm-Interp}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{M}_{\ell_0,\ldots,\ell_\kappa} \qquad \Gamma.🔒, \Delta \vdash t \Rightarrow T}{\Gamma \vdash t^s_\Delta \Rightarrow T^s_\Delta}$$

$$\textsc{Infer-Ascr}\ \frac{\Gamma \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash t \Leftarrow T}{\Gamma \vdash t : T \Rightarrow T} \qquad \textsc{Check-Inferred}\ \frac{\Gamma \vdash t \Rightarrow S \qquad \Gamma \vdash S \equiv T}{\Gamma \vdash t \Leftarrow T}$$

Fig. 7. Bidirectional term typing rules

In recent works [3] [1], a streamlined definition is given for parametricity interpretation of dependently-typed systems dubbed «external parametricity». Once again, this relational interpretation is definable in our system *internally* with the help of a relational model $\mathcal{U}_{\mathrm{para}}$. Since interpreted

functions do not depend on local context, we sidestep issues with relational context highlighted by Altenkirch and others [1], thus result of the interpretation is a proper proof of a free theorem.

4.2.4 Synthetic mathematics. As thoroughly discussed in a recent paper by Nawrocki and others [13], thanks to MLTT being an internal language of certain $(\infty, 1)$-categories, we can do *synthetic mathematics* inside it: prove a theorem inside MLTT about types; interpret it in a model of MLTT and yield a proof about objects in that model. It is easy to see that reinterpretation operator we have in MLTT-R is sufficient to do synthetic mathematics in it and immediately apply results interpreted in a suitable model.

5 Discussion & related work

To the best of our knowledge, we didn't find any research which explores the topic at hand, but the idea is certainly in the air, as illustrated by the following brilliant papers.

Original work by Lambek [9] sets the tone of the future research, establishing correspondence between type systems based on lambda calculus with various kinds of categories, starting with correspondence between simply typed lambda calculus and cartesian closed categories.

Conal Elliott's "Compiling to Categories" [6] shows how to interpret (some subset of) Haskell in arbitrary `Category` (again, definable in Haskell) via GHC compiler plugin. That paper is a direct inspiration for this one. There is an important difference, though: reinterpretation is internal to our type systems, unlocking new patterns in the usage of reinterpretation, including (but not limited to) universe polymorphism, external parametricity and more. However, Conal's approach is more flexible since his translation to categories requires desired category to only implement primitives actually used for translation of a particular term at hand.

In a recent paper "A Certifying Proof Assistant for Synthetic Mathematics in Lean", Wojciech Nawrocki and others [13] adapt Conal's approach to Lean's system with dependent types, using Categories-with-Families to represent internal models and employing Lean's powerful metaprogramming facilities instead of a compiler plugin. In doing so, they are able to provide a framework for doing synthetic mathematics in Lean, extracting proofs done in MLTT applied to more specific categories. Thus our results in section 4.2.4 are close to the results shown in their paper. To add to that, their work is already implemented in practical proof assistant. Yet, our type system involves no metaprogramming since reinterpretation is internal to it, and it handles conversion rule gracefully. In addition, the focus is a bit different, where we use reinterpretation to express other polymorphic features of a language.

Conor McBride's "Crude but effective stratification" [10, 11] (systemic treatment of this idea is given by Favonia and others [8]) introduces an eye-opening idea of using *displacement* as a lightweight form of universe polymorphism: to reuse term typed at a lower universe level, lift all levels in a term by adding a constant shift s. From the point of view advocated by this paper, you can see it as an act of reinterpreting an expression in an internal model of type system where universe at level i is interpreted as a universe at level $i + s$ (section 4.2.1).

In "Parametricity and Dependent Types" by Jean-Philippe Bernardy, Patrik Jansson and Ross Paterson [3], authors introduce a direct translation procedure to yield parametricity theorems and proofs from polymorphic definitions in a dependently typed system. The definition of «external parametricity» we follow, though, is given in "Internal Parametricity, without an Interval" by Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi and Michael Shulman [1]. Once again, we can reformulate the translation they give as reinterpretation in an internal model. Altenkirch and others noted an issue with this approach: if we apply the external translation to the judgment with nonempty context, context becomes parametric, too; that's the reason they add `rel` operation

and yield type system with *computational properties of higher observational type theory*. We sidestep the issue with context since reinterpretation is applied only to terms whose context is also «reinterpretable» and is not affected by the translation (section 4.2.3).

5.1 What we could've done differently

5.1.1 Modality of global elements. One could hope to salvage a situation by providing a "modality of global elements" which is a convenient relaxation of a notion of closed terms.

Definition 5.1. A dependently-typed system with a type of functions ($\to$) has a *global elements modality* $\Box$ if there is a one-hole term $\mathrm{nec}\ _$ and an atomic two-hole term $\mathrm{K}\ _\ _$ such that the following rules are admissible:

$$\frac{\cdot \vdash T\ \mathrm{type}}{\Gamma \vdash \Box T\ \mathrm{type}} \qquad \frac{\cdot \vdash t : T}{\Gamma \vdash \mathrm{nec}\ t : \Box T} \qquad \frac{\Gamma \vdash f : \Box\,(S \to T) \qquad \Gamma \vdash t : \Box S}{\Gamma \vdash K\ f\ t : \Box T}$$

And, in addition, the following reduction is admissible: $K\ (\mathrm{nec}\ \lambda x.t)\ (\mathrm{nec}\ s) \to^* \mathrm{nec}(t[x \coloneqq s])$.

The intuition behind the reduction is that, if we apply a "global function" to a "global argument", we should be able to "inline" this application, what this reduction precisely does.

Then, a corresponding weakening of computational reinterpretation can be defined as follows.

Definition 5.2 (Modal computational reinterpretation). We call a type operator $F[_]$ a modal computational reinterpretation of types if it is defined on boxed types (if $\Gamma \vdash \Box T\ \mathrm{Type}$, then $\Gamma \vdash F[T]\ \mathrm{Type}$), respects computation on types (if $T \to T'$, then $F[T] \to^* F[T']$) and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a modal computational reinterpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : \Box T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Maybe, this definition of reintrerpretation is "nice"? Regrettably, there's a no-go theorem which is similar to naïve version 2.2 which holds in this case:

Theorem 5.3 (No-go theorem of modal reinterpretation). *In a confluent type system with modal computational reinterpretation, the following should hold for arbitrary well-typed s and t:*

$$\mathsf{app}[\![\mathrm{nec}\ \lambda x.t]\!][\![\mathrm{nec}\ s]\!] \equiv [\![\mathrm{nec}(t[x \coloneqq s])]\!]$$

where $\equiv$ is definitional equality and app *is how $[\![_]\!]$ acts on K.*

Proof. By direct example. We are given well-formed types $\cdot \vdash S, T\ \mathrm{type}$ and well-typed terms $\cdot \vdash s : S$ and $\cdot, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : \Box(S \to T), y : \Box S \vdash [\![K\ x\ y]\!] : F[T]$$

Since K is atomic, term under interpretation is in normal form, hence the whole term computes to $\mathsf{app}[\![x]\!][\![y]\!]$. Substituting $\mathrm{nec}\ \lambda x.t$ for x and $\mathrm{nec}\ s$ for y, we get $\mathsf{app}[\![\mathrm{nec}\ \lambda x.t]\!][\![\mathrm{nec}\ s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![K\ (\mathrm{nec}\ \lambda x.t)\ (\mathrm{nec}\ s)]\!] : F[T]$. Since modal reinterpretation respects reduction, this reduces to $[\![\mathrm{nec}(t[x \coloneqq s])]\!]$.

In a confluent type system with functions, multi-step reduction commutes with substitution, therefore two terms are definitionally equivalent. □

6 Conclusions and further work

In retrospect, we managed to define two self-interpreting type systems, STLC-R and MLTT-R, meaning that they have a notion of their own «internal models» which they then can use to reinterpret their own terms inside such models. In doing so, such type systems get new features which

previously were defined as separate builtin quirks of a system. Basic metatheoretical properties are proven; for future work we have the following plans:

- Provide mechanized proofs of metatheoretical properties of systems;
- Provide implementation of MLTT-R in a practical proof assistant;
- Handle «partial» interpretation a-la Conal Elliott's "Compiling to Categories" [6];
- Make equality in MLTT-R more extensional, most probably by making type system closer to Homotopy type theory [16];
- Relax context constraints in reinterpretable terms.

References

[1] Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi, and Michael Shulman. 2024. Internal Parametricity, without an Interval. *Proc. ACM Program. Lang.* 8, POPL, Article 78 (Jan. 2024), 30 pages. doi:10.1145/3632920

[2] Thorsten Altenkirch and Ondrej Rypacek. 2012. A Syntactical Approach to Weak omega-Groupoids. In *Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 16)*, Patrick Cégielski and Arnaud Durand (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 16–30. doi:10.4230/LIPIcs.CSL.2012.16

[3] Jean-Philippe Bernardy, Patrik Jansson, and Ross Paterson. 2010. Parametricity and dependent types. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 345–356. doi:10.1145/1863543.1863592

[4] Simon Castellan, Pierre Clairambault, and Peter Dybjer. 2020. Categories with Families: Unityped, Simply Typed, and Dependently Typed. arXiv:1904.00827 [cs.LO] https://arxiv.org/abs/1904.00827

[5] Dominique Clément, Thierry Despeyroux, Gilles Kahn, and Joëlle Despeyroux. 1986. A simple applicative language: mini-ML. In *Proceedings of the 1986 ACM Conference on LISP and Functional Programming* (Cambridge, Massachusetts, USA) *(LFP '86)*. Association for Computing Machinery, New York, NY, USA, 13–27. doi:10.1145/319838.319847

[6] Conal Elliott. 2017. Compiling to categories. *Proc. ACM Program. Lang.* 1, ICFP, Article 27 (Aug. 2017), 27 pages. doi:10.1145/3110271

[7] Martin Hofmann and Thomas Streicher. 1994. The groupoid model refutes uniqueness of identity proofs. In *Proceedings of the Ninth Annual IEEE Symp. on Logic in Computer Science, LICS 1994* (Paris, France), Samson Abramsky (Ed.). IEEE Computer Society Press, 208–212.

[8] Kuen-Bang Hou (Favonia), Carlo Angiuli, and Reed Mullanix. 2023. An Order-Theoretic Analysis of Universe Polymorphism. *Proc. ACM Program. Lang.* 7, POPL, Article 57 (Jan. 2023), 27 pages. doi:10.1145/3571250

[9] Joachim Lambek. 1980. From λ-calculus to cartesian closed categories. In *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*, J. P. Seldin and J. R. Hindley (Eds.). Academic Press, London, 375–402.

[10] Conor McBride. 2011. Crude but Effective Stratification. https://mazzo.li/epilogue/index.html%3Fp=857&cpage=1.html

[11] Conor McBride. 2012. Crude but Effective Stratification. https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf

[12] Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. doi:10.1016/0890-5401(91)90052-4 Selections from 1989 IEEE Symposium on Logic in Computer Science.

[13] Wojciech Nawrocki, Joseph Hua, Mario Carneiro, Yiming Xu, Spencer Woolfson, Shuge Rong, Sina Hazratpour, and Steve Awodey. 2026. A Certifying Proof Assistant for Synthetic Mathematics in Lean. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) *(CPP '26)*. Association for Computing Machinery, New York, NY, USA, 88–103. doi:10.1145/3779031.3779087

[14] John C Reynolds. 1983. Types, abstraction and parametric polymorphism. In *Information Processing 83*, R. E. A. Mason (Ed.). North-Holland, 513–523.

[15] Gregor Richards. 2025. Proof of the Church-Rosser Theorem. https://student.cs.uwaterloo.ca/~cs442/W25/extras/c-r-thm-proof.pdf

[16] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. https://homotopytypetheory.org/book, Institute for Advanced Study.